```
          {"from": 70000,"to": 80000},
          {"from": 80000,"to": 120000}
        ]
      }
    }
  }
}
```

We define a `range` aggregation bucket type with a set of custom ranges. Once the query is executed, the resulting buckets show the keys with the custom bucket ranges and the number of documents for each range:

```
"aggregations" : {
  "range_countries" : {
    "buckets" : [{
      "key" : "*-60000.0",
      "to" : 60000.0,
      "doc_count" : 1
    },{
      "key" : "60000.0-70000.0",
      "from" : 60000.0,
      "to" : 70000.0,
      "doc_count" : 0
    },{
      "key" : "70000.0-80000.0",
      "from" : 70000.0,
      "to" : 80000.0,
      "doc_count" : 2
    },{
      "key" : "80000.0-120000.0",
      "from" : 80000.0,
      "to" : 120000.0,
      "doc_count" : 3
    }]
  }
}
```

The results indicate that one country has up to 60,000 casualties, three countries a number of casualties between 80,000 and 12,0000, and so on.

We can perform a rich set of aggregations on our data using out-of-the-box statistical functions. We run through these in chapter 13; until then, hold tight.

We've barely scratched the surface of what Elasticsearch offers; we explore much more functionality in the coming chapters. For now, it's time to call it a wrap. In the next chapter, we examine the architecture of Elasticsearch, the mechanics of search, its moving parts, and much more.

Summary

- Elasticsearch provides a set of document APIs for indexing data, and Kibana's Dev Tools console helps with writing indexing queries for persistence.
- To retrieve a document using a single document API, we issue a `GET` command on the index with an ID (`GET <index_name>/_doc/<ID>`).